

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>Н2</u>	<u>Программная инженерия и интеллектуальные системы</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Проектирование архитектуры программных средств</u>	
		наименование дисциплины

УЧЕБНО–ПРАКТИЧЕСКАЯ РАБОТА

2

номер задания (при наличии)

ДИАГРАММА ПРЕЦЕДЕНТОВ UML

при наличии указать тему учебно–практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О729Б

Барышников О.А.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Зими́на Д.В.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

г. Санкт–Петербург

20 26 г.

СОДЕРЖАНИЕ

1 Диаграмма вариантов использования	3
2 Описание диаграммы вариантов использования	4

1 Диаграмма вариантов использования

На рисунке 1 изображена UML-диаграмма вариантов использования.

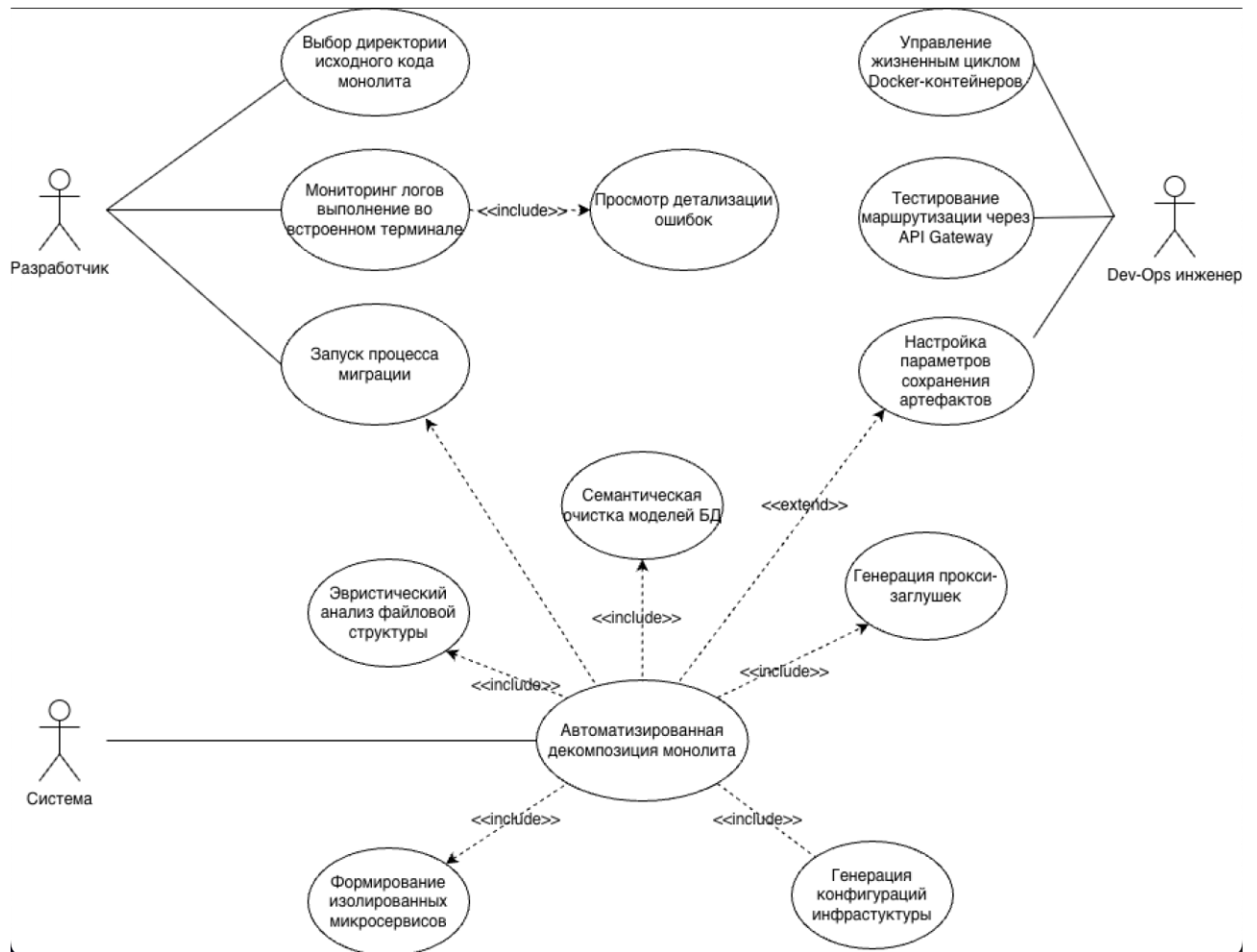


Рисунок 1 – Диаграмма вариантов использования

2 Описание диаграммы вариантов использования

Диаграмма вариантов использования описывает взаимодействие пользователей с системой «Программный инструмент автоматизированной декомпозиции монолитных веб-приложений».

В системе выделены два актора:

- разработчик, выполняющий загрузку исходного кода монолита, инициализацию миграции и анализ логов генерации;
- DevOps-инженер, осуществляющий настройку выходной конфигурации и управление сгенерированной Docker-инфраструктурой.

Разработчик взаимодействует со следующими вариантами использования:

- “Выбор директории исходного кода монолита”;
- “Запуск процесса миграции”;
- “Мониторинг логов выполнения во встроенном терминале”.

Вариант использования «Просмотр детализации ошибок» не вызывается разработчиком напрямую, а включается (<<include>>) в состав варианта использования:

- «Мониторинг логов выполнения во встроенном терминале».

Таким образом, детальная информация о системных сбоях (например, о циклических импортах) становится доступной только в контексте наблюдения за потоком вывода (stdout) процесса генерации.

DevOps-инженер взаимодействует со следующими вариантами использования:

- “Настройка параметров сохранения артефактов”;
- “Управление жизненным циклом Docker-контейнеров (Up/Down)”;
- “Тестирование маршрутизации через API Gateway”.

При этом вариант использования «Настройка алгоритмов эвристического поиска» в данной диаграмме отсутствует, так как эти алгоритмы заложены в ядро системы жестко и не настраиваются пользователем напрямую.

Центральным вариантом использования системы является «Автоматизированная декомпозиция монолита», который реализует основной программный функционал ядра АММ-Docker.

Данный вариант использования включает (<<include>>) последовательность этапов обработки исходного кода:

- “Эвристический анализ файловой структуры (Scanner)”;
- “Семантическая очистка моделей БД (удаление ForeignKey)”;
- “Генерация прокси-заглушек (Stub)”;
- “Формирование изолированных микросервисов (Generator)”;
- “Генерация конфигураций инфраструктуры (Docker, Nginx)”.

Эти этапы отражают строгий конвейерный принцип трансформации исходного кода из монолитной архитектуры в микросервисную.

Вариант использования «Автоматизированная декомпозиция монолита» расширяется (<<extend>>) вариантом использования «Настройка параметров сохранения артефактов», выполняемым DevOps-инженером. Это означает, что в процессе работы системы может применяться пользовательская (кастомная) директория для вывода результата, однако сама настройка пути выполняется отдельно и не является обязательной (при ее отсутствии система использует выходную директорию по умолчанию).

Таким образом, диаграмма отражает функциональное разделение ролей при работе с инструментом АММ-Docker, описывает конвейерную структуру лексического анализа и генерации, а также показывает зависимость процесса от пользовательской конфигурации путей.